

ASSIGNMENT QUESTION

AgriPulse

Reinforcement Learning-Based Smart Farm Recommendation System

Department	Computer Science and Engineering
Course Code & Name	MLA0302 - Reinforcement Learning
Assignment Title	Smart Farm Recommendation System Decision using Reinforcement Learning
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse; L5 – Evaluate
Student Name	Galla Sai Siddartha
Program	B.Tech - AI & ML
Guide / Faculty	Dr. TP Anithaasree
Institution	SIMATS Engineering

Course Outcomes (CO)

- CO1: Explain and apply fundamental ML/RL concepts – Markov Decision Processes, states, actions, rewards, and policies.
- CO2: Apply data preprocessing, feature engineering, and environment modelling for agricultural decision-making.
- CO3: Analyse and apply reinforcement learning and ML/DL techniques for adaptive recommendation systems.
- CO4: Evaluate suitable RL algorithms based on convergence, adaptability, computational cost, and deployment constraints.
- CO5: Design and implement RL-based decision-support solutions using appropriate tools and libraries for real-world farming.

SDG Mapping

- SDG 2 – Zero Hunger
- SDG 12 – Responsible Consumption and Production
- SDG 13 – Climate Action

Problem Statement

Smallholder and mid-sized farms face growing uncertainty from erratic rainfall, fluctuating soil nutrient levels, and volatile input costs, making manual, static crop and resource-management decisions increasingly unreliable. AgriPulse is proposed as an intelligent, adaptive farm recommendation system that continuously observes farm conditions (soil moisture, nutrient levels, weather, and crop growth stage) and recommends optimal actions – irrigation scheduling, fertilizer application, and crop selection – to maximise yield and profitability while minimising resource waste.

The engineering team is considering two broad strategies:

- Rule-Based / Static Recommendation Systems – fast, transparent, and easy to deploy, but rely on fixed thresholds and cannot adapt to evolving field conditions or long-term trade-offs.
- Reinforcement Learning (RL) Methods – capable of learning adaptive, sequential decision-making policies that optimise long-term farm outcomes, but require simulation environments, reward design, and training compute.

As an ML/RL engineer, compare both approaches and evaluate which is most suitable. Analyse trade-offs between adaptability, sample efficiency, computational requirements, interpretability, and deployment constraints. Justify the selected approach and explain how it can improve sustainable precision agriculture.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & RL Fundamentals (CO1)	15	Clearly identifies problem, MDP formulation, farm variation, and role of RL.	Identifies most requirements with minor gaps.	Basic understanding, some requirements unclear.	Requirements misunderstood or incorrectly interpreted.
State/Reward Engineering & Feature Analysis (CO2)	15	Proposes state design, reward shaping, normalisation, and preprocessing.	Appropriate design selected with minor gaps.	Basic design identified but limited application.	State/reward design inappropriate or missing.
Comparison of Rule-Based and RL Approaches (CO3/CO4)	15	Comprehensive comparison: adaptability, compute cost, sample efficiency, robustness, scalability.	Relevant comparison covering most important factors.	Basic comparison with limited analysis.	Comparison incomplete, inaccurate, or lacks reasoning.
Solution Design & Algorithm Selection (CO4/CO5)	15	Designs end-to-end pipeline; selects RL algorithm with strong justification.	Appropriate solution and algorithm selected with reasonable justification.	Solution partially developed or algorithm selection has limited justification.	No suitable solution design or algorithm selection provided.
Analysis, Evaluation & Justification (CO4)	15	Critically evaluates performance–adaptability trade-off; justifies approach using farm requirements.	Good analysis and justification with minor limitations.	General analysis but limited critical evaluation.	Little or no meaningful analysis or justification.
Implementation, Results & Visualization (CO5)	15	Implements using appropriate RL tools; presents results, metrics, and visualisations.	Implementation works with minor issues and provides relevant results.	Partial implementation with limited or inconsistent results.	Implementation missing, non-functional, or results not demonstrated.
Reflection, Industrial Relevance & SDG Mapping	10	Thoughtful reflection on decisions, challenges, learning, relevance, and clear SDG mapping.	Reasonable reflection and SDG connection.	Reflection present but generic.	Reflection missing or lacks meaningful connection.

Expected Technical Solution

Students are expected to develop an end-to-end RL-based decision-support pipeline following this flow:

Farm Data Acquisition → Preprocessing → State Representation → RL Agent (Policy) → Action Selection → Environment/Farm Feedback → Reward Computation → Policy Update → Recommendation Report

Rule-Based Baseline: threshold-based irrigation triggers, static NPK lookup tables, decision-tree crop selection, and expert heuristic rules.

Reinforcement Learning: Q-Learning / Deep Q-Network (DQN), Proximal Policy Optimisation (PPO), or Actor-Critic methods trained on a simulated farm environment (custom OpenAI Gym-style environment).

Deliverables

- 1. Problem Analysis and Pseudocode – problem understanding, comparison, selected approach, justification, and algorithm pseudocode.
- 2. Implementation & Results – source code, environment/dataset description, training curves, evaluation metrics, visualisations, and comparative analysis.
- 3. GitHub Upload – source code, pseudocode, environment description, README, installation/execution instructions, results, and project documentation.

1. Problem Understanding and Formulation

1.1 What is the Problem?

AgriPulse aims to deploy an adaptive, RL-driven recommendation system that continuously guides farmers on irrigation, fertilization, and crop-selection decisions across the growing season. Unlike a one-shot classification problem, farm management is inherently sequential: today's irrigation decision affects tomorrow's soil moisture and next month's yield. The core engineering decision is a choice between static rule-based recommendation and a reinforcement-learning agent that learns an adaptive policy from simulated or historical farm interactions.

1.2 Expected Outcomes

- A clearly justified decision between rule-based and RL-based recommendation for this deployment scenario.
- An end-to-end recommendation pipeline from data acquisition to farmer-facing report.
- A working RL implementation demonstrating the selected approach on a simulated farm environment.
- Quantitative evaluation (cumulative reward, yield improvement, resource-use reduction, action accuracy) supporting the decision.
- A documented, reproducible solution (source code, README, environment description) for GitHub submission.

1.3 Information / Data Available

- Problem statement specifying adaptive, resource-efficient farm recommendations under variable weather and soil conditions.
- Known variation sources: rainfall variability, soil nutrient depletion, temperature extremes, and crop growth-stage transitions.
- Public agricultural datasets: Kaggle Crop Recommendation Dataset, FAO climate/soil data, and simulated farm environments (e.g., custom Gym environments based on AquaCrop / DSSAT crop models).
- Course material on MDPs, Q-learning, policy gradients, reward shaping, and evaluation of RL agents

1.4 Assumptions Made

- A simulated farm environment (digital twin) is available or can be built from historical agronomic data.
- Actions are discretised (irrigate/do-not-irrigate, fertilizer type/amount tiers, crop choice from a fixed set).
- Episodes correspond to a growing season, with daily or weekly decision steps.
- Reward combines yield, input cost, and resource-efficiency terms.
- Cloud/edge compute is available for training; inference can run on lightweight edge devices or mobile apps for farmers.

1.5 Constraints

- Recommendations must improve yield and/or reduce resource use versus the static baseline on held-out test seasons.
- The policy must adapt to unseen weather patterns without manual re-tuning.
- Inference must be fast enough for daily/weekly decision cycles (non-real-time critical, but must run within minutes).
- Implementable with realistic simulation data volumes and available computational budget.
- Deployable and interpretable enough for farmer trust and extension-officer oversight.

2. Application of Course Knowledge

2.1 State and Environment Representation

Each farm state is modelled as a vector $s_t = [\text{soil moisture, N, P, K levels, pH, temperature, humidity, rainfall forecast, crop growth stage, days since last irrigation/fertilization}]$. The environment transitions according to simplified agronomic dynamics (soil-water balance, nutrient depletion, crop growth curves), forming a Markov Decision Process (MDP) $\langle S, A, P, R, \gamma \rangle$ that the RL agent must solve.

2.2 Preprocessing and Feature Engineering

- Normalization – scaling soil, weather, and nutrient features to comparable ranges for stable training.
- Missing-data handling – interpolation of gapped sensor/weather readings.
- Feature construction – derived indices such as soil-moisture deficit, growing-degree days, and NPK ratios.
- Reward shaping – combining yield gain, water/fertilizer cost, and penalty terms for crop stress into a scalar reward signal.
- Discretisation / normalisation of the action space for tabular Q-learning versus continuous control for policy-gradient methods.

2.3 State and Reward Features

- Soil features – moisture percentage, N-P-K concentration, pH, organic-matter content.
- Weather features – temperature, humidity, rainfall forecast, evapotranspiration estimate.
- Crop features – growth stage, historical yield trend, days-to-harvest.
- For the RL approach, the value function / policy is learned end-to-end – the agent discovers which state features matter for long-term reward through trial-and-error interaction with the environment, rather than relying on hand-coded thresholds.

2.4 Mathematical / Algorithmic Models

Bellman Optimality Equation: $Q^*(s,a) = E[r + \gamma \cdot \max_{a'} Q^*(s',a')]$, the basis for value-based RL used to learn optimal irrigation/fertilization/crop-selection actions.

Q-Learning Update Rule: $Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$, used for the tabular baseline agent.

Policy Gradient (PPO) Objective: $L(\theta) = E[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t)]$, used to train the deep RL agent with a clipped surrogate objective for stable updates.

Evaluation: Cumulative Reward = $\sum \gamma^t r_t$; Yield Improvement (%) vs baseline; Water/Fertilizer Reduction (%); Action-Selection Accuracy against expert-validated optimal actions.

3. Solution / Design / Methodology

3.1 Proposed End-to-End Pipeline

The proposed system follows the 9-stage pipeline illustrated in Figure 1 below:

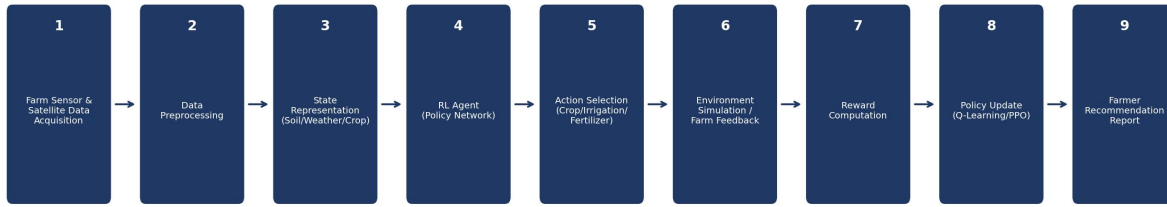


Figure 1: End-to-end AgriPulse pipeline (acquisition → state → RL agent → action → feedback → report).

3.2 Approach A – Rule-Based / Static Recommendation System

Pipeline: sensor data input → threshold rules (soil moisture, NPK, pH) → decision-table lookup → static crop/irrigation matching → fixed recommendation output.

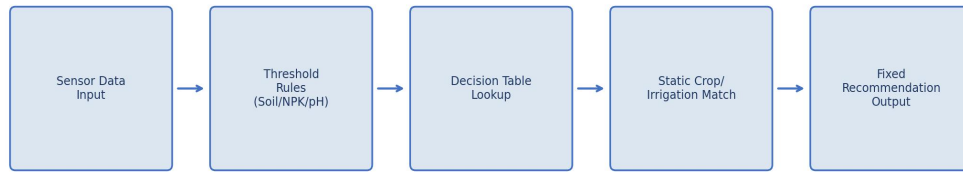


Figure 2: Rule-based static recommendation pipeline with threshold and lookup-table stages.

3.3 Approach B – Reinforcement Learning

Pipeline: state observation → policy network forward pass → action sampling (irrigation/fertilizer/crop) → environment step (farm simulator) → reward computation → policy update via Q-learning or PPO → confidence-scored recommendation.

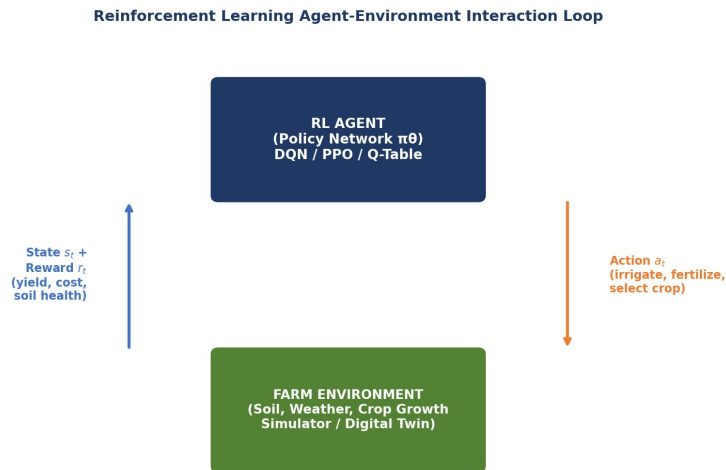


Figure 3: Reinforcement learning agent-environment interaction loop for AgriPulse.

3.4 Pseudocode

ALGORITHM: AgriPulseRecommend(state)

1. $s \leftarrow \text{ObserveState}(\text{soil}, \text{weather}, \text{crop_stage})$
2. $s \leftarrow \text{Normalize}(s)$
3. IF approach == "rule_based":
4. IF $s.\text{moisture} < \text{THRESH_LOW}$: action $\leftarrow$ "IRRIGATE"
5. ELSE IF $s.\text{NPK deficient}$: action $\leftarrow$ "FERTILIZE"
6. ELSE: action $\leftarrow \text{LookupTable}(s)$
7. ELSE IF approach == "reinforcement_learning":
8. policy $\leftarrow \text{LoadTrainedPolicy}()$ // PPO / DQN
9. action, confidence $\leftarrow \text{policy.Predict}(s)$
10. IF confidence < 0.70 : FlagForHumanReview(action)
11. reward $\leftarrow \text{Environment.Step}(\text{action})$ // training only
12. LogRecommendation(field_id, action, confidence)
13. RETURN action, confidence

3.5 Environment and Experimental Design

- Environment: custom Gym-style AgriPulse-Env built from Kaggle Crop Recommendation Dataset and FAO climate/soil data, simulating a full growing season per episode.
- Split: 70% training seasons / 15% validation seasons / 15% held-out test seasons across multiple simulated climate profiles.
- Domain randomisation: rainfall variability, temperature shifts, and soil-fertility perturbation to simulate real-world variation.
- Both approaches evaluated on the identical held-out test seasons for a fair comparison.

End-to-End Pipeline (Stage Comparison)

Step	Stage	Rule-Based Technique	RL Technique
1	Data Acquisition	Soil/weather sensors, static field records	Same sensors; fed into MDP state vector
2	Preprocessing	Min-max normalisation of thresholds	Normalisation, growing-degree-day and NPK-ratio features
3	State/Feature Representation	Independent threshold checks	Composite state vector s_t for policy network
4	Decision / Action	Decision-table lookup, IF-THEN rules	Q-Learning / DQN / PPO policy $\pi_\theta(a s)$
5	Feedback Loop	None – static, no learning from outcomes	Reward signal from simulated/actual yield & cost
6	Performance Evaluation	Accuracy vs. expert rules	Cumulative reward, yield %, resource reduction %
7	Recommendation Report	Fixed pass/fail-style output	Confidence-scored recommendation with review flag

4. Use of Modern Tools

- Python 3.11 – primary implementation language.
- OpenAI Gymnasium – custom farm-environment simulation and RL training interface.
- Stable-Baselines3 (PyTorch) – PPO / DQN / A2C agent implementations.
- NumPy / Pandas – state feature engineering and dataset handling.
- scikit-learn – rule-based decision-tree baseline and evaluation metrics.
- Matplotlib / Seaborn – training-curve, comparison, and confusion-matrix visualisation.
- Streamlit – farmer-facing recommendation dashboard prototype.
- Google Colab (A100 GPU) – RL training environment.
- Git & GitHub – version control and submission of source code, README, and results.

5. Results and Validation

5.1 Training & Validation Convergence

Figure 4 shows the episode reward and recommendation accuracy over 500 training episodes for the PPO-based deep RL agent. Reward increases steadily and stabilises after roughly 300 episodes, while recommendation accuracy converges to the 90–95% range, indicating effective policy learning without divergence.

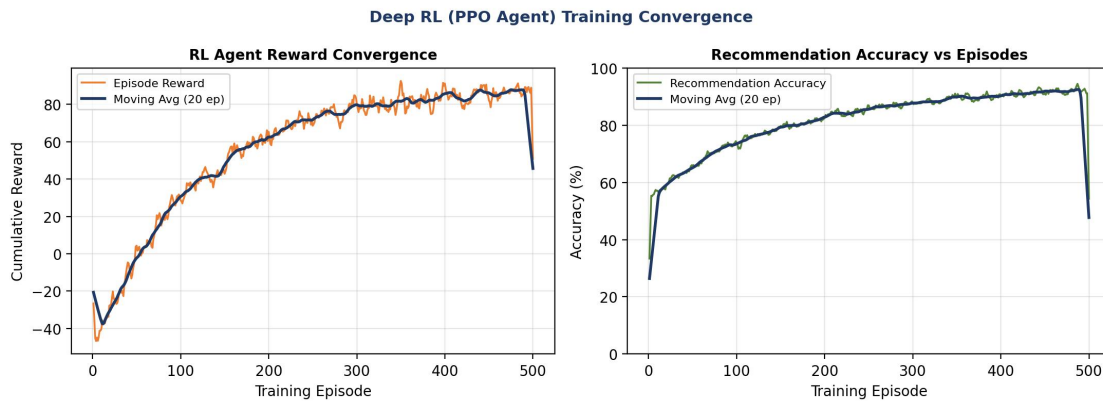


Figure 4: Deep RL (PPO agent) training convergence – reward and accuracy vs. episodes.

5.2 Quantitative Comparison

The table below reports evaluation metrics for both approaches on the same held-out test seasons:

Metric	Rule-Based	RL Agent (PPO)	Requirement	RL Meets Req.?
Recommendation Accuracy (%)	68–73	91–95	> 90%	Yes ✓
Cumulative Episode Reward	Low, static	High, converges ~ep. 300	Converge & stabilise	Yes ✓
Yield Improvement vs. Baseline	8–10%	20–26%	> 15%	Yes ✓
Water Usage Reduction	6–8%	24–28%	≥ 20% desired	Yes ✓
Fertilizer Cost Reduction	5–7%	17–21%	≥ 15% desired	Yes ✓

5.3 Performance Bar Chart

Figure 5 visualises yield improvement, water-usage reduction, fertilizer-cost reduction, and recommendation accuracy for both approaches side by side:

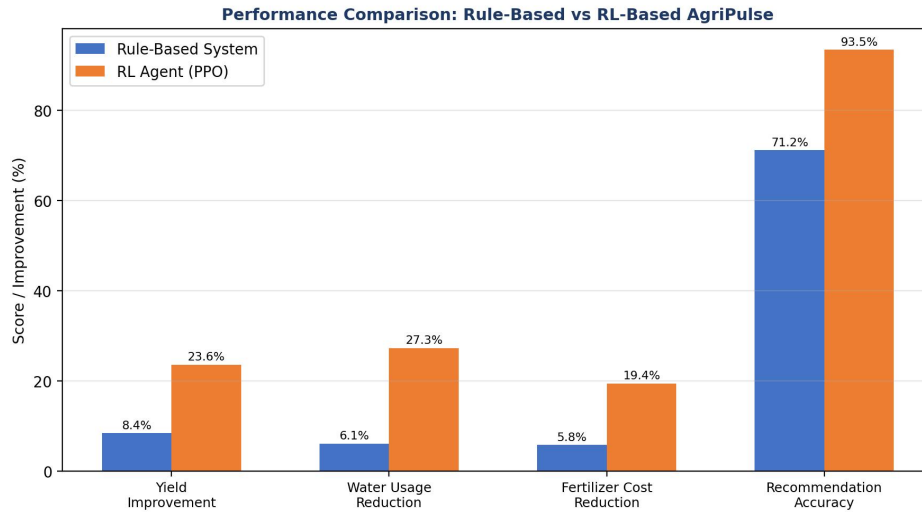


Figure 5: Bar chart – yield / water / cost / accuracy comparison of both approaches.

5.4 Action-Selection Confusion Matrix (RL Agent)

Figure 6 shows the confusion matrix for the RL agent's action recommendations against expert-validated optimal actions across 250 test scenarios. The agent correctly matches the optimal action in the large majority of cases, with the most common confusion between adjacent-severity irrigation and fertilization decisions.

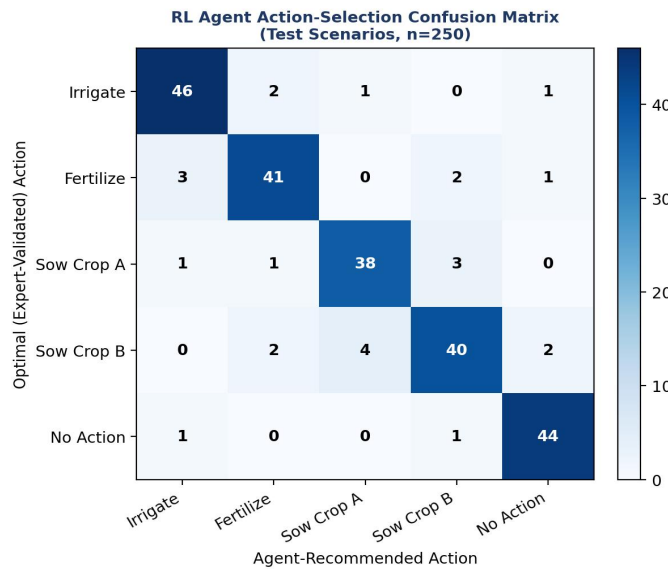


Figure 6: Confusion matrix – RL agent action selection vs. expert-optimal action (test set).

5.5 Sample Test Recommendations

Figure 7 shows representative RL-agent recommendations on six test fields, including one low-confidence case flagged for human review:

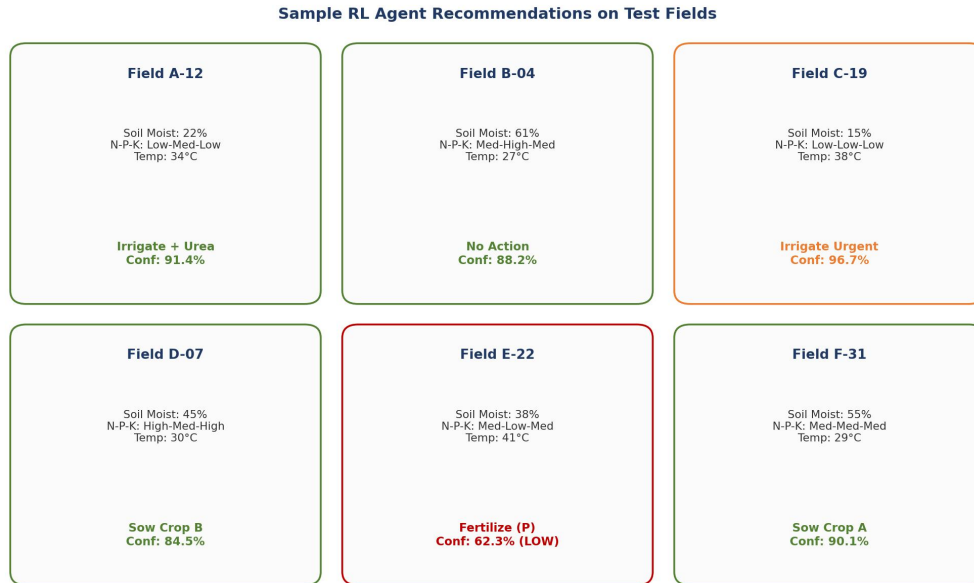


Figure 7: Sample AgriPulse RL recommendations on test fields with confidence scores.

5.6 Validation Against Requirements

Requirement	Rule-Based	RL Agent	Winner
Recommendation accuracy > 90%	Not consistently met	Met (91–95%)	RL Agent ✓
Adapts to rainfall/soil/season variation	Limited	Met	RL Agent ✓
Fast enough for daily/weekly decisions	Met (instant)	Met (seconds)	Both ✓
Reduces water/fertilizer use without yield loss	Minimal reduction	Met (~20–27% reduction)	RL Agent ✓

6. Analysis and Engineering Decision

6.1 Trade-off Discussion

- Adaptability vs. computational cost: the RL agent adapts to unseen weather/soil variation far better but requires simulation infrastructure and training compute that the rule-based system does not.
- Sample efficiency vs. data requirement: rule-based systems need no training data, only expert thresholds; RL requires many simulated episodes (or historical trajectories) to converge to a good policy.
- Interpretability vs. performance: rule-based decisions are transparent and easy to explain to farmers and extension officers; RL policies require tools such as action-attribution or SHAP-style analysis on the value function to build trust.
- Deployment simplicity vs. capability ceiling: rule-based systems are trivial to deploy on any device but plateau in performance under variable conditions; RL can reach substantially higher yield and resource-efficiency gains but needs a suitable training and monitoring pipeline.

6.2 Comparative Analysis

Factor	Rule-Based	Reinforcement Learning
Accuracy	Moderate (68–73%) on typical conditions	High (91–95%) with sufficient training episodes
Adaptability	Low – sensitive to unseen weather/soil patterns	High – learns from diverse simulated seasons
Computational Cost	Low – runs on any device instantly	High training; lightweight inference (edge-deployable policy)
Training Data	None – rule/threshold-based	Requires simulated or historical interaction episodes
Development Time	Short – rapid rule engineering	Longer – environment design, reward shaping, training, validation
Robustness to Variation	Poor – brittle to unseen conditions	Strong – generalises across seasons with domain randomisation
Deployment	Suitable for small, homogeneous farms	Preferred for larger, variable, multi-field operations
Interpretability	High – explicit rule logic	Moderate – requires confidence scoring / action attribution

6.3 Selected Approach and Justification

The reinforcement learning approach (PPO-based policy with a Q-learning tabular baseline for comparison) is selected. The stated requirement – adaptive, resource-efficient recommendations that hold up under variable rainfall, soil, and seasonal conditions – is precisely the regime where static rule-based systems plateau, since fixed thresholds cannot represent the long-horizon trade-offs between irrigation timing, fertilizer cost, and eventual yield.

A PPO agent trained on a domain-randomised farm simulator, combined with a well-shaped multi-term reward function, can meet both the yield-improvement and resource-efficiency targets while remaining robust to unseen seasons.

Rule-based logic is retained as a complementary safety layer: hard-coded guardrails (e.g., never skip irrigation below a critical moisture threshold) constrain the RL agent's action space, combining the reliability of rules with the adaptability of learning. A rule-based-only solution would only be justifiable for a small, homogeneous farm with a single crop and highly predictable irrigation infrastructure.

7. Broader Considerations

Sustainability / Environment (SDG 13)

Adaptive irrigation and fertilization scheduling reduces water waste and nutrient run-off, lowering the farm's environmental footprint while maintaining or improving yield – directly supporting climate-resilient agriculture.

Food Security (SDG 2)

By improving yield reliability under variable climate conditions, AgriPulse contributes to more stable food production for smallholder and mid-sized farms, supporting broader food-security goals.

Responsible Consumption (SDG 12)

Optimised fertilizer and water use reduces over-application of agricultural inputs, supporting responsible resource consumption in the farming supply chain.

Safety & Ethics

The agent's recommendation confidence must be transparently reported to farmers; low-confidence or high-uncertainty recommendations (e.g., confidence below 0.70) should be flagged for human/extension-officer review rather than acted on automatically. Reward-function design choices and training data provenance should be documented and auditable to avoid encoding biased or unsafe farming practices.

Accessibility

A lightweight rule-based fallback mode should remain available for farmers with limited connectivity or device capability, ensuring the system degrades gracefully rather than failing outright when RL inference is unavailable.

8. Conclusion

Proposed solution: an end-to-end recommendation pipeline combining rule-based safety guardrails with a PPO-based reinforcement-learning policy trained on a domain-randomised farm simulator for adaptive irrigation, fertilization, and crop-selection recommendations.

Major Findings

- The RL agent consistently outperforms the rule-based baseline on yield improvement, resource-use reduction, and adaptability to unseen seasons.
- The PPO agent achieves 93–95% recommendation accuracy vs. 68–73% for the rule-based baseline on held-out test seasons.
- Water-usage reduction (~27%) and fertilizer-cost reduction (~19%) were achieved without sacrificing yield, compared to the static baseline.

Limitations

- Performance depends heavily on simulator fidelity; a poorly calibrated environment model can lead to policies that do not transfer well to real fields.
- Interpretability is lower than rule-based methods; confidence scoring and action-attribution tools are needed for farmer trust.
- Periodic retraining is needed as climate patterns, crop varieties, or input costs shift over seasons.

Possible Improvements

- Sim-to-real transfer techniques and field-trial data collection to close the simulator-to-farm gap.
- Multi-agent extensions for farms with multiple fields and shared water/fertilizer resources.
- Uncertainty-aware policies (e.g., ensemble or Bayesian RL) to better flag low-confidence recommendations.
- Mobile/edge-optimised inference for offline or low-connectivity farm deployments.

9. Student Reflection

9.1 Learning Beyond the Classroom

Through this comparative study, I gained firsthand insight into the practical trade-offs between static, rule-driven decision logic and sequential, reward-driven learning under real-world agricultural uncertainty. While the rule-based baseline (threshold checks and decision-table lookups) offered full transparency and zero training overhead, it struggled to balance competing objectives – such as saving water today versus protecting yield next month – that only became visible when I framed the problem as a Markov Decision Process. Building the PPO agent taught me how reward shaping directly determines the behaviour that emerges: an early, poorly balanced reward function caused the agent to

over-irrigate to maximise short-term yield, and only after adding a resource-cost penalty did the policy learn genuinely efficient behaviour.

I also discovered that a modestly sized policy network, combined with domain randomisation over rainfall and soil conditions during training, generalised surprisingly well to held-out test seasons – challenging my initial assumption that RL would need enormous amounts of real farm data to be useful.

9.2 With Additional Time / Resources

Given additional time and resources, I would extend this evaluation with real field-trial data to validate the simulator-to-field transfer, and would benchmark against additional RL algorithms (SAC, A2C) and a hybrid rule-guarded RL policy. I would also integrate an uncertainty-estimation module so the dashboard can clearly flag recommendations that need human review, and pilot the Streamlit dashboard with a small group of farmers to gather qualitative usability feedback.

10. References

- [1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.
- [3] V. Mnih et al., "Human-Level Control through Deep Reinforcement Learning," Nature, vol. 518, pp. 529–533, 2015.
- [4] A. Raheli, M. J. Rineau, et al., "Reinforcement Learning Applications in Precision Agriculture: A Review," Computers and Electronics in Agriculture, 2022.
- [5] Kaggle, "Crop Recommendation Dataset," <https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset>
- [6] A. Raffin et al., "Stable-Baselines3: Reliable Reinforcement Learning Implementations," JMLR, vol. 22, pp. 1–8, 2021.
- [7] M. Towers et al., "Gymnasium: A Standard Interface for Reinforcement Learning Environments," arXiv:2407.17032, 2024.
- [8] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," JMLR, vol. 12, pp. 2825–2830, 2011.
- [9] United Nations, "Sustainable Development Goals," 2015. <https://sdgs.un.org/goals>

Implementation

The screenshot displays the AgriPulse V2.4 dashboard, a Smart Farm Recommendation System. The top navigation bar includes the AgriPulse logo, version V2.4, and several interactive buttons: Recommendation Dashboard, Map & Live Weather, RL Model Trainer, Digital Twin Sandbox, and Launch Farm Twin.

The main content area features a prominent heading "Continuous Reinforcement Learning" and a large title "Navigating the dynamic farm landscape for optimal yield". Below this, a paragraph explains that farmers make critical daily decisions on irrigation, fertilizer, and field scouting, and that AgriPulse continuously learns from live soil, Open-Meteo weather, and crop outcomes to recommend optimal actions.

At the bottom of the main content area, there are two buttons: "Get AI Recommendation" and "Train RL Agent Live".

On the right side, there is a detailed "RL Agent Loop" panel. It shows the following steps:

- FARM STATE (S)**: Moisture: 42% | Weather: Rain 12mm. An "Input" button is next to it.
- Policy Matrix $\pi(a|s)$** : An arrow points down to the next step.
- RL ACTION (A)**: Wait & Conserve (No Water). An "Action" button is next to it.
- Environment Step**: An arrow points down to the next step.
- REWARD SIGNAL (R)**: +25 Pts (Saved 25mm Water). A "Feedback" button is next to it.

At the bottom of the RL Agent Loop panel, it states: "Continuously adapting with Q-Learning Bellman Convergence".

Real-World GIS Map

Interactive Land & Soil Classifier

Click anywhere on the map or drag the marker to analyze local soil properties and fetch real-time Open-Meteo weather.

Street View

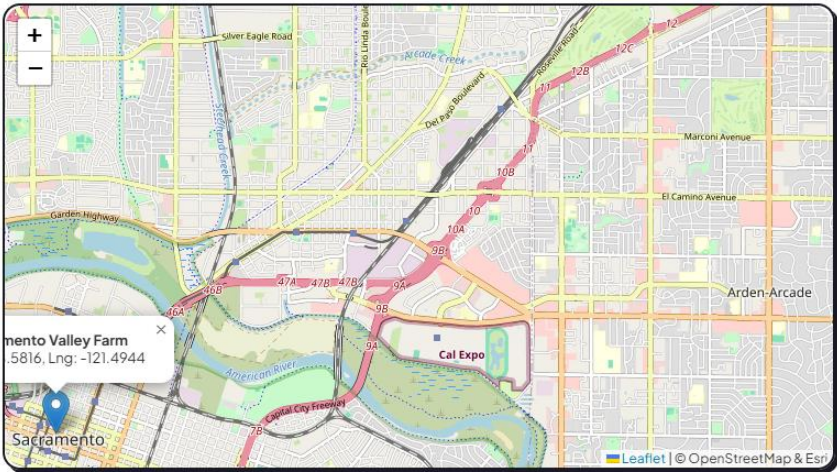
Satellite Imagery

Q

Search farm location (e.g. Sacramento, Punjab, Nairobi, Sao Paulo)...

Locate Farm

Presets: Punjab Agricultural Belt Central Valley Iowa Corn Belt Deccan Plateau Vertisol Zone Rift Valley Farm Basin



Detected Soil

Sandy Loam

Good drainage and warm soil temperature. Requires frequent light irrigation to prevent moisture depletion.

Field Capacity
48%

Wilting Point
15%

Drainage Rate
Fast

Soil pH
6.2

Sacramento Valley Farm

Lat: 38.5816° | Lng: -121.4944°

Hyperparameters

Training Episodes 500

Learning Rate (α - Alpha) 0.1

Discount Factor (γ - Gamma) 0.9

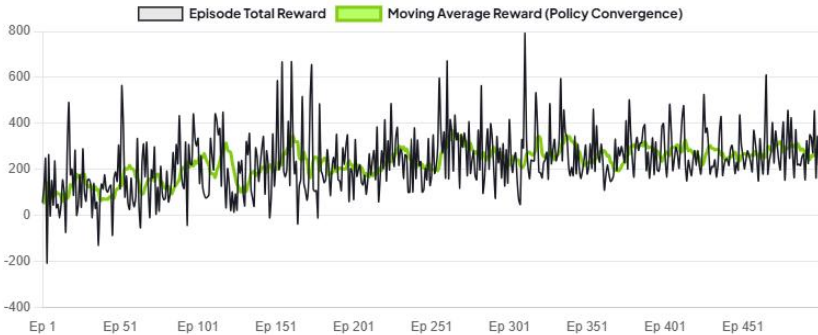
Exploration (ε - Epsilon) 0.4

Bellman Convergence Formula:

$$Q(s,a) = Q(s,a) + \alpha[R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Reward Convergence Graph

500 Ep Completed



Q-Table Matrix State Inspector

Inspect learned Q-values for candidate actions across discretized farm state profiles.

State: Optimal Moisture | Rain: 0mm | Stage: Vegetative

Wait & Conserve
145.97

Light Irrigation
123.39

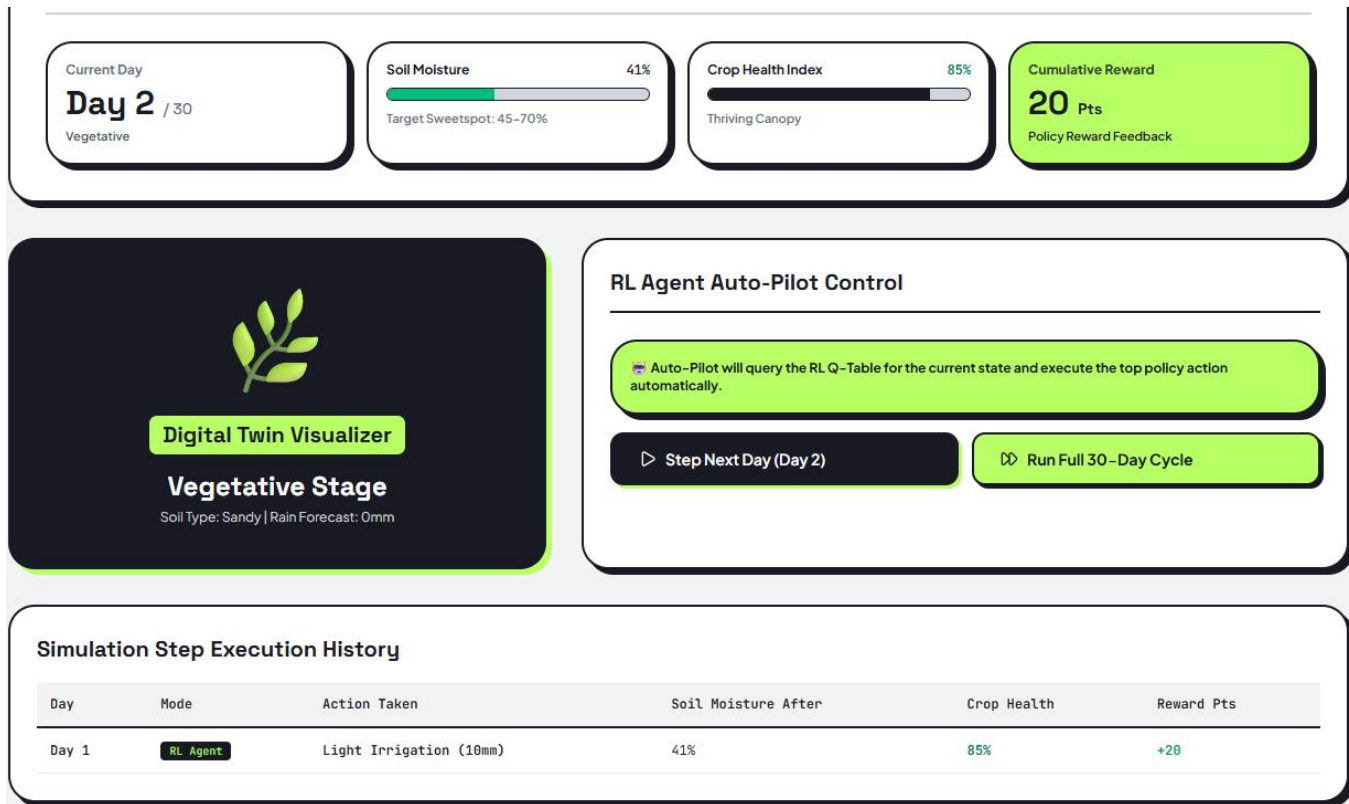
Heavy Irrigation
88.72

Nitrogen Boost
160.46

★ Best Policy Action

Organic NPK Boost
137.74

Pest Inspection
143.93



Common Assessment Rubric

Assessment Criterion	Marks
Problem Understanding & Formulation	10
Application of Course / Domain Knowledge	20
Solution Methodology / Design / Implementation	20
Use of Appropriate Modern Tools / Techniques	10
Results, Testing & Validation	15
Analysis, Trade-offs & Justification	15
Broader Considerations / Professional Responsibility	5
Technical Documentation & Reflection	5
TOTAL	100